

Singleton lab



Objetivo

Modelar e implementar um sistema simples utilizando o padrão de projeto **Singleton**, conforme visto em aula.



Contexto

Um sistema de venda de veículos precisa manter os dados do vendedor responsável pelo sistema.

Como existe apenas um vendedor principal configurado no sistema, não deve ser possível criar vários objetos Vendedor.

A classe Vendedor deve possuir uma única instância, acessada por meio de um método estático.



Estrutura esperada

O sistema deve seguir o padrão Singleton, com:

Classe Singleton

- Vendedor

Classe de teste

- Programa



Tarefa

Implementar o sistema em Java respeitando a arquitetura do padrão Singleton.



Restrições

- Não permitir `new Vendedor()` fora da própria classe.
- O construtor de Vendedor deve ser `private`.
- A instância única deve ser armazenada em um atributo `static`.
- O acesso ao vendedor deve ocorrer pelo método `getInstance()`.
- Usar nomes em português.



Elementos obrigatórios

Classe

Vendedor

Atributos

```
private String nome;
```

```
private String endereco;
```

```
private String email;
```

```
private static Vendedor instancia;
```

Métodos

```
private Vendedor()
public static Vendedor getInstance()
public void setNome(String nome)
public void setEndereco(String endereco)
public void setEmail(String email)
public void exibir()
```

Observação

O método `getInstance()` deve criar o objeto apenas se ele ainda não existir. Caso contrário, deve retornar a instância já criada.



Especificação da classe de teste

A classe Programa deve demonstrar o funcionamento do Singleton da seguinte forma:

1. Obter uma primeira referência para o vendedor usando `Vendedor.getInstance()`.
2. Alterar os dados do vendedor: nome, endereço e email.
3. Obter uma segunda referência para o vendedor usando novamente `Vendedor.getInstance()`.
4. Exibir os dados do vendedor usando a segunda referência.
5. Comparar as duas referências para mostrar que elas apontam para o mesmo objeto.



Saída esperada

Nome: Vendedor Auto

Endereço: Uberlândia

Email: vendedor@auto.com

true



Interpretação

- Vendedor representa o Singleton do sistema.
- O construtor privado impede a criação direta de objetos com `new`.
- `getInstance()` controla o acesso à única instância.
- As duas referências obtidas apontam para o mesmo objeto.
- Por isso, a comparação entre elas retorna `true`.



Trabalho de casa

O código **não deve ser enviado**. Ele deverá ser preparado pelos alunos e apresentado apenas durante a aula de laboratório.

Questões

1. Faça o diagrama de classe UML da solução.

O diagrama deve conter a classe Vendedor, seus atributos, seus métodos e a indicação de que o construtor é privado.

2. Por que o construtor da classe Vendedor deve ser private?
3. Qual é o papel do atributo private static Vendedor instancia?
4. Por que o método getInstance() precisa ser static?
5. O que aconteceria se o método getInstance() sempre criasse um novo objeto?
6. Por que a comparação entre as duas referências deve retornar true?
7. Em que tipo de situação real faria sentido existir apenas um objeto responsável por uma configuração ou controle do sistema?
8. Qual seria a diferença entre usar Singleton e simplesmente usar apenas atributos e métodos static?